QA Run Report: run_2ad92c6dab

■ Executive Quality Summary

Overall Quality Score: `25/100`
Current Status: `FAILED`

■■ Core Metrics Breakdown

Reliability Score: `80/100` (Static & Runtime Stability)

Validation Score: `20/100` (Business Logic Coverage)

Hallucination Risk: `0/100` (Ungrounded Logic Detectors)

Resilience Health: `0/100` (Error Handling & Retries)

■ Critical Security & Logic Findings

[CRITICAL] Application Initialization Failure: The app crashed during import. This usually means top-level dependencies or env vars are missing.

Error: unable to open database file

[HIGH] Missing required field: edges: The uploaded automation.json is missing the top-level `edges` field.

[HIGH] Workflow edges are missing: A node/edge workflow was uploaded without `edges`.

[MEDIUM] Workflow node type is not in the approved registry: Node `collector` declares unsupported type `file_collector`.

[MEDIUM] Workflow node type is not in the approved registry: Node `planner` declares unsupported type `llm_planner`.

[MEDIUM] Workflow node type is not in the approved registry: Node `router` declares unsupported type `tool_router`.

[MEDIUM] Workflow node type is not in the approved registry: Node `executor` declares unsupported type `python_runner`.

[MEDIUM] Workflow node type is not in the approved registry: Node `memory` declares unsupported type `vector_memory`.

[MEDIUM] Workflow node type is not in the approved registry: Node `repair` declares unsupported type `self_healer`.

[MEDIUM] Workflow node type is not in the approved registry: Node `approval` declares unsupported type `approval_gate`.

[MEDIUM] Workflow node type is not in the approved registry: Node `notifier` declares unsupported type `email_sender`.

[MEDIUM] Workflow node type is not in the approved registry: Node `finalizer` declares unsupported type `finalizer`.

[HIGH] Missing validation layer: The workflow has no explicit validation node or rule after execution steps.

[MEDIUM] Retry policy is missing: The workflow does not declare retry attempts, delays, or backoff strategy.

■ Autonomous Failure Explainer

Root Cause Analysis

The application failed to initialize due to missing dependencies or environment variables, specifically the 'DB_URL' environment variable, which is required to connect to the SQLite database.

User Impact: If this issue is not resolved before deployment to production, users will experience a complete application failure, resulting in a '500 Internal Server Error' or a similar error message, indicating that the application is not available or functioning as expected.

Recommended Fix: Set the 'DB_URL' environment variable to a valid SQLite database URL, such as 'sqlite:///example.db', before running the application. Alternatively, modify the application code to handle the case where 'DB_URL' is not set, for example, by using a try-except block to catch and handle the 'unable to open database file' error.

■■ Autonomous Repair Strategies

Strategy: Safe Fallback for DB_URL and API_SECRET

Safety Score: 98.0%

Rationale: Providing safe fallback values for environment variables prevents application crashes due to missing or incorrect environment variables

Suggested Code Patch:

```
import os
import sqlite3
from fastapi import FastAPI
DATABASE_URL = os.getenv("DB_URL", "sqlite:///:memory:")
API_SECRET = os.getenv("API_SECRET", "default_secret")
conn = sqlite3.connect(DATABASE_URL)
cursor = conn.cursor()
app = FastAPI()

@app.get("/")
def root():
```

```
    return {"status": "running"}
```

Strategy: Error Handling for SQLite Connection

Safety Score: 92.0%

Rationale: Adding error handling for SQLite connection prevents application crashes due to connection issues

Suggested Code Patch:

```
import os
import sqlite3
from fastapi import FastAPI
DATABASE_URL = os.getenv("DB_URL", "sqlite:///:memory:")
API_SECRET = os.getenv("API_SECRET", "default_secret")
try:
    conn = sqlite3.connect(DATABASE_URL)
    cursor = conn.cursor()
except sqlite3.Error as e:
    print(f"Error connecting to database: {e}")
app = FastAPI()

@app.get("/")
def root():
    return {"status": "running"}
```

Strategy: Type Checking and Division by Zero Prevention

Safety Score: 90.0%

Rationale: Adding type checking and division by zero prevention prevents application crashes due to type errors or division by zero

Suggested Code Patch:

```
import os
import sqlite3
from fastapi import FastAPI
DATABASE_URL = os.getenv("DB_URL", "sqlite:///:memory:")
API_SECRET = os.getenv("API_SECRET", "default_secret")
if not isinstance(DATABASE_URL, str):
    raise TypeError("DATABASE_URL must be a string")
try:
    conn = sqlite3.connect(DATABASE_URL)
    cursor = conn.cursor()
except sqlite3.Error as e:
    print(f"Error connecting to database: {e}")
app = FastAPI()

@app.get("/")
def root():
    return {"status": "running"}
```

■ Agent Execution Timeline

`05:40:16` **reflection**: Agent reflection active (running)

`05:40:16` **reflection**: Transitioned to reflection (running)

`05:40:18` **failure_explainer**: The application failed to initialize due to missing dependencies or environment variables, specifically the 'DB_URL' environment variable, which is required to connect to the SQLite database. (success)

`05:40:18` **memory_retriever**: Agent memory_retriever active (running)

`05:40:18` **memory_retriever**: Transitioned to memory_retriever (running)

`05:40:20` **memory_retriever**: Retrieved 3 past fixes. (success)

`05:40:20` **self_heal_router**: Agent self_heal_router active (running)

`05:40:20` **self_heal_router**: Transitioned to self_heal_router (running)

`05:40:23` **self_heal_router**: Proposed 3 fixes (success)

`05:40:23` **memory_writer**: Agent memory_writer active (running)

`05:40:23` **memory_writer**: Transitioned to memory_writer (running)

`05:40:28` **retry_or_replan**: Agent retry_or_replan active (running)

`05:40:28` **retry_or_replan**: Transitioned to retry_or_replan (running)

`05:40:28` **finalizer**: Agent finalizer active (running)

`05:40:28` **finalizer**: Transitioned to finalizer (running)